Université Mohammed Premier
Faculté des Sciences
Département d'Informatique

Année universitaire 2022-2023
Filière SMI – S6
Programmation Système

Examen de la session ordinaire - Durée (1h30)

Exercice 1 (Entourer la/les bonne(s) réponse(s) : plusieurs réponses correctes sont possibles)) (5 pts)

1. Dans le cas où un thread appelle la fonction exec(), les autres threads :
   a) Vont continuer leur exécution normale
   b) Vont terminer leur exécution
   c) Vont appeler la fonction main()
   d) Vont envoyer un signal au thread qui appelé exec()

2. Une variable locale d'un thread est accessible par :
   a) Seulement le thread de cette variable
   b) Tous les threads du même programme

3. Int pthread_detach(pthread t) est une fonction qui
   a) permet de détacher un thread sans pouvoir le ré-attacher par la suite
   b) permet de détacher un thread avec possibilité de ré-attachement par la suite
   c) permet de libérer les ressources du thread concerné

4. On veut qu'un processus envoie à lui-même un signal, on doit donc écrire :
   a) raise(int sig)
   b) kill(getpid(), sig)
   c) alarm(getpid())
   d) Un processus ne peut jamais s'envoyer un signal, on doit impérativement avoir un autre processus

5. On veut installer un handler pour le signal « SIGKILL ». Pour ce faire, on peut utiliser :
   a) La primitive signal
   b) La primitive sigaction
   c) Le signal SIGKILL n'existe pas
   d) On ne peut pas installer un handler pour le signal SIGKILL

Exercice 2 (Questions de compréhension : Donnez une réponse brève et bien précise) (3 pts)

1. Qu'est-ce que c'est la commutation ou le changement de contexte ?
……
……
……

2. Pourquoi faire une synchronisation des threads ? Donner les 3 solutions de synchronisation
……
……
……
……

3. C'est quoi la différence entre signal() et sigaction()
……
……
……

Exercice 3 : (5 pts)

1. Combien de processus sont créés par ce programme ? **(1pt)**

```
int main() {
if(fork()==0){
   if(fork()){
        for (i=0; i<2; i++)
                fork();
   } }}
```

2. Donner l'arbre de processus crées **(1 pt)**
3. Détecter les erreurs dans le programme suivant **(1.5 pt)**

```c
#include <stdio.h>
char message[7]= "Bonjour";
main(){

    int p[2];
    int pipe(int[2]);
    if(pipe(p) == -1) {
        fprintf(stderr,"Erreur d'ouverture du pipe");
        exit(1);
    }
    if (fork()==0) {
        char c;
        while (read(p[0]), &c, 1)!=0)
            printf("%c",c);
        exit(0);
    }
    else{
        write(p[1],message,7);
        exit(0);
    }

}
```

4. Qu'est-ce qu'on doit ajouter pour corriger ces erreurs ? **(1.5 pt)**

Exercice 4 : (7 pts)

On désire simuler un mécanisme client-serveur **simplifié** de réservation de places. Il y a 100 places qui sont représentées par un tableau d'entiers. Place[i] vaut 0 si la place est libre, et vaut le numéro du client si Place[i] est occupée. Le thread principal attend en entrée la saisie du nombre de places à réserver par le client. Si cet entier est supérieur ou égal à 0, il indique le nombre de places à réserver, sinon il indique l'arrêt des demandes de réservation et provoque l'affichage du tableau des places. Après chaque demande de réservation, le thread initial crée un thread pour réserver les places demandées et ensuite se met en attente (le thread créé). Les clients sont numérotés par ordre d'arrivée.

T.A.F : Implémentez un programme en C en utilisant les threads et les synchronisations nécessaires pour simuler ce mécanisme.